

StochLean - Measure Convergence, Projective Laws & Canonical Construction

Work Handoff Design v0.1 | Klenke Ch.13-14 provenance, functional ownership, external Kolmogorov-extension integration

Normative Status

This document is a Work implementation contract. Klenke is the coverage/provenance baseline, not the directory structure. All Lean files and milestones are organized by mathematical function and dependency. The global no-duplicate, no-semantic-cheating, trust-boundary, CI, and longevity rules inherited from the earlier StochLean handoffs remain fully binding.

0. Terminology Resolution: Klenke "Borel space" = Mathlib StandardBorelSpace

Klenke Definition 8.35 calls a measurable space $(E, \mathcal{E})$ a Borel space when it is measurably isomorphic to $(B, \mathcal{B}(B))$ for some Borel subset B of $\mathbb{R}$. Mathlib StandardBorelSpace is the standard equivalent formulation: the measurable structure is the Borel sigma-algebra of some compatible Polish topology. Mathlib also proves that every StandardBorelSpace is measurably equivalent to a measurable subset of $\mathbb{R}$ (exists_subset_real_measurableEquiv), matching Klenke's definition directly. Conversely, a measurable space isomorphic to a Borel subset of $\mathbb{R}$ is a standard Borel space by the standard Borel-isomorphism theorem.

Normative mapping: Klenke "Borel space" $\rightarrow$ StandardBorelSpace. Do not create a new Klenke-specific class. Do not confuse this with Mathlib BorelSpace, which means that an already chosen topology has the given measurable structure as its Borel sigma-algebra.

1. Global Hard Rules (Inherited, Non-Negotiable)

Coverage baseline. Klenke 3rd ed. is a completeness floor. Every relevant formal Definition/Lemma/Theorem/Corollary/load-bearing exercise receives a ledger status, even when no code is added.

Functional organization. Never create textbook-chapter-named Lean modules merely to mirror Klenke. Ownership follows mathematical functionality and dependency.

Duplicate gate. Before creating any declaration: audit pinned Mathlib $\rightarrow$ complete local StochLean tree $\rightarrow$ approved external libraries. If canonical functionality exists, REUSE it. Thin BRIDGE is allowed only for a documented ergonomic/semantic gap.

No proof holes / no semantic cheating. No sorry, admit, project axioms, impossible/vacuous types, conclusion-shaped assumptions, theorem hidden in structures, surrogate false models, arbitrary totalization outside the mathematical domain, or strengthened/weakened statements that silently change the theorem.

Semantic distinctions. Keep weak/vague/setwise convergence distinct; keep f.d.d. law, coordinate-product law, and topological path-space law distinct; keep StandardBorelSpace distinct from BorelSpace; keep projective consistency distinct from path regularity.

Trust boundary. Every external dependency must have exact source, exact revision, license, compile status, axiom/dependency audit, and approved/unapproved status. Moving master is forbidden for trusted builds.

Source corrections/generalizations. If a Lean theorem is more general than Klenke (e.g. StandardBorel vs a chosen Polish topology), record it explicitly as a proved generalization. Do not silently alter the source statement.

Longevity/upstream. When functionality is upstreamed to Mathlib, rerun duplicate audit, replace facade implementation, remove obsolete dependencies, and preserve downstream public theorem statements whenever possible.

CI completion. Compilation alone is insufficient. Required: no sorry/admit/custom axiom; semantic regression tests; dependency/axiom checks; minimal-import checks; public-API leak checks; source/coverage ledger closure.

2. Package Scope and Dependency Boundary

Build order is fixed:

Part I: Measure Convergence & Tightness $\rightarrow$ Part II: Product / Kernel / Projective Construction	
Function	Canonical owner / directive
Weak convergence of probability measures	Mathlib topology on ProbabilityMeasure; REUSE.
Vague convergence	Generic measure theory; AUDIT-FIRST, BUILD only if a real gap survives.
Portmanteau / tightness / Prokhorov	Mathlib; REUSE/BRIDGE only.
Convergence in distribution / Slutsky	Mathlib; REUSE.
Empirical measure object / de Finetti theorem	Previous DiscreteMatingale/Exchangeability package; this package only supplies weak/tightness route.

Function	Canonical owner / directive
Product measurable spaces / cylinders / finite products	Mathlib; REUSE.
Kernel composition / Ionescu-Tulcea	Mathlib; REUSE/BRIDGE.
Projective-family vocabulary	Mathlib IsProjectiveMeasureFamily / IsProjectiveLimit; REUSE.
Arbitrary-index Kolmogorov extension	Primary construction milestone; audited external proof + thin StandardBorel facade, otherwise minimal port/build.
Canonical process	Raw coordinate function only; no structure.
Markov semigroups	Future Markov package.
Convolution semigroups / stationary independent increments	Future Levy/SII package.
Continuous/cadlag path regularity and path-space topology	Future Path Regularity / Functional Weak Convergence packages.

3. Part I - Measure Convergence & Tightness

Canonical decision 1 (fixed): weak convergence of probability measures is ordinary Tendsto in the existing ProbabilityMeasure topology. StochLean must not create a second WeaklyConverges predicate.

Klenke source / function	Status	Work directive
13.1 topology primer / Polish spaces	REUSE	Use Mathlib topology, Polish, compactness, regularity infrastructure.
13.9-13.11 separating classes	REUSE/BRIDGE	Use existing bounded-continuous / Lipschitz separation. Add only a generic convergence-determining-class theorem if exact functionality is missing.
13.12 weak convergence	REUSE	Use topology/Tendsto on finite/probability measures; no new predicate.
13.12 vague convergence	AUDIT-FIRST	Keep distinct from weak convergence; for Radon/locally finite measures. BUILD only if canonical gap remains.
13.14 functional-analytic vs probability weak convergence note	SEMANTIC RULE	Never conflate Banach weak convergence with weak convergence of measures.
13.16 Portmanteau	REUSE/BRIDGE	Reuse Mathlib Portmanteau open/closed/null-frontier criteria.
13.17 convergence in distribution	REUSE	Use TendstoInDistribution = weak convergence of laws.
13.18 Slutsky	REUSE	Use Mathlib convergence-in-distribution machinery.
13.19 convergence in probability -> distribution	REUSE	Converse is false; keep notions separate.
Continuous mapping theorem, limit-a.e. continuity form	AUDIT-FIRST/BRIDGE	Global-continuity theorem is not enough if Klenke-level a.e.-continuity version is absent.
13.21 real CDF characterization	REUSE/BRIDGE	Bridge to existing law/CDF infrastructure; do not define a separate CDF weak-convergence notion.
13.26 tightness	REUSE	Use MeasureTheory.IsTightMeasureSet.
13.29 Prokhorov	REUSE/BRIDGE	Use Mathlib Prokhorov when present on active pin; if pin lacks it, backport canonical code rather than private recreation.
13.30-13.31 subsequential/vague compactness	REUSE/BRIDGE	Use Prokhorov/tightness APIs.
13.34 separating+tight => weak	AUDIT-FIRST/BRIDGE	Generic convergence-determining class theorem if exact gap survives audit.
13.35 weak <-> vague + mass/tightness	AUDIT-FIRST/BRIDGE	Important bridge for random measures/PPP.
13.4 empirical-measure weak/tightness route	BUILD/BRIDGE	Consume the previous package empirical-measure object; do not create a second object/CDF.
13.4 second de Finetti proof	APPLICATION	Must reuse the already canonical public de Finetti theorem; no deFinetti2 API.

Part I mandatory distinctions

- vague convergence alone does not imply weak probability convergence (mass may escape);
- weak convergence does not imply setwise convergence; null-frontier hypotheses matter;
- convergence in probability implies convergence in distribution, but not conversely;
- continuous mapping with merely measurable/discontinuous maps is invalid unless the limit law gives zero mass to the discontinuity set;
- weak convergence of empirical measures must reuse the previous empirical measure and de Finetti APIs.

4. Part II - Product, Kernel, and Projective Construction

Klenke source / function	Status	Work directive
14.1 product measurable spaces / coordinate maps / cylinders	REUSE	Use Mathlib Pi/product measurable-space and cylinder infrastructure.
Countable Polish product Borel theorem	REUSE	No textbook-specific recreation.
Finite product measures / Tonelli / Fubini	REUSE	Mathlib measure/integral infrastructure.
Convolution in Ch.14	REUSE	Reuse the single canonical measure convolution fixed earlier; never define a second convolution.
14.23 kernel integral measurability	REUSE	Use ProbabilityTheory.Kernel integration/measurability API.
14.34 finite history-dependent experiments	REUSE/BRIDGE	Use kernel product/composition.
14.35 Ionescu-Tulcea	REUSE/AUDIT-FIRST	Use Mathlib trajectory construction; if active pin lacks it, backport canonical implementation.
14.37 consistency / projective family	REUSE	Use IsProjectiveMeasureFamily / IsProjectiveLimit only.
14.38 countable-index extension	SPECIALIZATION	Once general StandardBorel KET exists, do not maintain a second implementation.
14.39 arbitrary-index Kolmogorov extension	KEY MILESTONE	Audited external Polish proof + StandardBorel facade; minimal port/build only if audit fails.
14.40 arbitrary independent product law	REUSE/COROLLARY	Acceptance theorem/example from the projective extension.
projective family -> canonical coordinate process f.d.d. recovery	BUILD/BRIDGE if absent	Exact finite-dimensional-law bridge only; no new process structure.
14.42-14.48 Markov semigroup/process construction	DEFER	Next Markov package.
14.49-14.50 convolution semigroup / stationary independent increments	DEFER	Levy/SII package.

5. Kolmogorov Extension - Declaration-Level Design

Canonical decision 2 (fixed): arbitrary-index Kolmogorov extension for StandardBorelSpace is the true construction milestone of this package. The preferred proof source is an audited exact revision of RemyDegenne/kolmogorov_extension4, wrapped by a thin StochLean facade. Fresh reconstruction is a last resort.

5.1 External proof source and trust state

Candidate source: RemyDegenne/kolmogorov_extension4. Design-time inspected master commit: 7d76e184c3d2138a2741baf923b57e9a01b9cf25 (2026-07-24). License: Apache-2.0. The repository contains an arbitrary-index projective-limit construction under Polish-space hypotheses and uses Mathlib projective-family/cylinder infrastructure. It remains EXTERNAL-CHECK until the exact Work environment passes compile, statement, axiom/dependency, no-sorry/admit, license, and provenance audits.

Hard rule: never depend on moving master. Pin an exact external commit SHA. If the external audit fails, try compatibility adaptation first, then a minimal audited port, and only then a fresh BUILD.

5.2 StandardBorel facade

The public StochLean theorem must use only measurable-space-level StandardBorelSpace assumptions. Internally, use Mathlib upgradeStandardBorel to install a compatible Polish topology and call the audited Polish KET. The chosen topology is an implementation detail and must not leak into the public signature.

```
Provisional facade shape: noncomputable def MeasureTheory.projectiveLimitOfStandardBorel {ι : Type*} {α : ι -> Type*} [forall i, MeasurableSpace (α i)] [forall i, StandardBorelSpace (α i)] (P : forall J : Finset ι, Measure (forall j : J, α j)) [forall J, IsFiniteMeasure (P J)] (hP : IsProjectiveMeasureFamily P) : Measure (forall i, α i)
```

```
Required correctness theorem: theorem isProjectiveLimit_projectiveLimitOfStandardBorel ... : IsProjectiveLimit (projectiveLimitOfStandardBorel P hP) P
```

5.3 Probability and uniqueness layers

- Keep the base constructor at finite-measure generality. Probability is a typeclass corollary when every finite marginal is a probability measure.
- Do not add [Nonempty ι]. The empty-index case must work; the product over an empty index is a singleton and P_empty determines the mass.
- Use existing IsProjectiveLimit.isProbabilityMeasure rather than recomputing total mass.
- Expose uniqueness only by calling IsProjectiveLimit.unique; do not repeat the cylinder pi-system proof.

- An exists-unique theorem form may be added as a corollary if it improves downstream usage; it must not create another constructor.

5.4 Canonical coordinate process bridge

Do not define a CanonicalProcess structure. On the product sample space, the process is the raw coordinate family $\omega \mapsto \omega_i$. The only potentially useful StochLean bridge is the finite-dimensional recovery theorem: for every finite J , the pushforward under $J.\text{restrict}$ equals P_J . This should reduce directly to `IsProjectiveLimit` and existing `FiniteDimensionalLaws.lean`.

Semantic boundary: Kolmogorov extension constructs a probability law on the coordinate-product sigma-algebra. It does not supply continuous paths, cadlag paths, joint path measurability, or a topological path-space law.

6. External Audit and Fallback Policy

Check	Pass criterion / action
Revision pin	Exact commit SHA recorded; no moving branch/tag.
Compatibility	Builds under the actual StochLean Lean/Mathlib pin or under an explicitly approved synchronized pin.
Statement audit	Projective-limit theorem has the required arbitrary-index and finite-measure semantics; no hidden extra assumptions.
Axiom audit	No project-specific/custom axioms; <code>#print axioms/dependency closure</code> is acceptable.
Proof-hole audit	No <code>sorry/admit/unsafe</code> semantic shortcut in trusted theorem closure.
License/provenance	Apache-2.0 and source attribution recorded.
StandardBorel coverage	Thin facade proves the public StandardBorel theorem without leaking a chosen topology.
Empty index	No Nonempty index assumption in StochLean public probability result.
Failure response	Compatibility adaptation -> minimal audited port -> fresh BUILD last.

7. Functional Module Ownership

Possible modules are ownership boundaries, not a file-creation checklist:

```
ForMathlib/ MeasureTheory/Measure/VagueConvergence.lean # only if real gap
MeasureTheory/Measure/ConvergenceDetermining.lean # only if exact gap
MeasureTheory/Constructions/KolmogorovExtension.lean # facade / port only if needed
Probability/MeasureConvergence/CDF.lean # bridges only
MeasureConvergence/Empirical.lean # weak/tightness bridges only
Process/ProjectiveLaw.lean # f.d.d. / projective-limit bridges
Process/Construction/Projective.lean # high-level construction facade only
```

Do not create local mirrors named WeakConvergence, Portmanteau, Prokhorov, ProductMeasure, Kernel, or IonescuTulcea when the functionality is already canonical in Mathlib.

8. Work Implementation Order

Phase	Required work
A	Pinned Mathlib -> complete local StochLean (including Ch01-08, ProcessCore, DiscreteMatingale package) -> approved external duplicate/API audit.
B	Fix canonical weak-probability topology; map Portmanteau, TendstoInDistribution, tightness, Prokhorov. Do not recreate them.
C	Close only real gaps in vague convergence, a.e.-continuous mapping, convergence-determining classes, and weak-vague bridges.
D	Reuse prior empirical-measure/de Finetti APIs and formalize only the Ch.13.4 weak/tightness route; no duplicate empirical object/CDF/de Finetti.
E	Audit/reuse products, cylinders, finite products, Kernel composition, Ionescu-Tulcea.
F	Fix <code>IsProjectiveMeasureFamily</code> / <code>IsProjectiveLimit</code> as the only projective vocabulary; countable extension becomes a specialization once general KET exists.
G0	Recheck pinned Mathlib + full local tree for newly upstreamed KET before external integration.
G1	Pin exact kolmogorov_extension4 revision and compile in the actual Work toolchain.
G2	Axiom/dependency/no-sorry/statement/license audit; only then promote to AUDITED-REUSE.
G3	Implement thin StandardBorel facade using <code>upgradeStandardBorel</code> + audited external Polish KET.
G4	Add probability/uniqueness/f.d.d. bridges; no Nonempty ι assumption.
G5	Run semantic regressions, minimal imports, public-API-leak audit; then release downstream KET blockers.
H	Final coverage, source mapping, trust boundary, CI, longevity audit.

Blocking rule: before G5 passes, downstream Brownian/Markov/Levy/PPP packages may treat arbitrary KET as a blocker but may not create temporary duplicate implementations.

9. Mandatory Semantic Regression Suite

1. δ_n on $\mathbb{R}$ converges vaguely to 0 but not weakly as probability measures;
2. $\delta_{1/n} \rightarrow \delta_0$ weakly, but masses of $\{0\}$ do not converge;
3. continuous mapping cannot be used at a limit-law discontinuity point of positive mass;
4. convergence in probability $\rightarrow$ distribution, while the converse fails;
5. the family $\{\delta_n\}$ is not tight;
6. vague convergence with mass escape fails weak convergence unless mass/tightness conditions are added;
7. empirical-measure weak convergence uses the previous canonical empirical-measure object; real empirical CDF bridge recovers the earlier CDF;
8. coordinate maps on product spaces are measurable and cylinders generate the product sigma-algebra;
9. projectivity direction is correct: restricting P_J for $L \subset J$ gives P_L ;
10. explicit projective families with at least three coordinates recover every prescribed finite marginal;
11. empty index I : unique probability projective limit exists; public API has no Nonempty I assumption;
12. singleton index and finite index recovery tests;
13. arbitrary product marginals recover independent coordinates (Klenke 14.40 acceptance);
14. an abstract `StandardBorelSpace` with no user-provided natural topology can call the facade;
15. public facade signatures contain neither `PolishSpace` assumptions nor `KolmogorovExtension4` namespace references;
16. different compatible Polish upgrades produce the same resulting measure by projective-limit uniqueness;
17. output is only a product-sigma-algebra law; no theorem derives continuous/cadlag paths;
18. `Mathlib Probability/Process/Kolmogorov.lean` is rejected as evidence for Kolmogorov extension (it is Kolmogorov-Chentsov infrastructure);
19. finite projective measures do not automatically produce a probability instance;
20. when all finite marginals are probabilities, probability follows through existing `IsProjectiveLimit.isProbabilityMeasure`;
21. external dependency must be exact-revision pinned;
22. Ch.13.4 alternative de Finetti proof reuses the same existing public de Finetti theorem;
23. Markov semigroup and convolution-semigroup APIs remain outside this package.

10. Completion / Acceptance Criteria

- Every Klenke-relevant item in the ledger has a final status with source mapping.
- No duplicate weak-convergence, projective-family, kernel, product-measure, empirical-measure, or de Finetti objects are introduced.
- The `StandardBorel` facade is theorem-equivalent to Klenke 14.39 and does not expose implementation topology.
- Arbitrary-index KET is either audited-reused at an exact external revision or implemented by the documented fallback path.
- All mandatory semantic regressions pass.
- No sorry/admit/custom axiom; trusted dependency closure is audited.
- Minimal imports and public API do not leak external namespace details.
- Downstream packages can depend on one stable projective-extension interface, allowing future Mathlib upstream replacement without theorem-statement churn.

11. Source / Provenance Notes

Primary textbook source: Achim Klenke, *Probability Theory: A Comprehensive Course*, 3rd ed., Springer, 2020. Relevant provenance: Ch.13 (convergence of measures) and Ch.14 (product spaces, kernels, Ionescu-Tulcea, projective consistency, Kolmogorov extension). Terminology bridge: Klenke Def.8.35 "Borel space" is the standard Borel notion.

Mathlib reconnaissance used during design: `StandardBorelSpace` and `upgradeStandardBorel`; `ProbabilityMeasure` weak topology / Portmanteau; `TendstoInDistribution`; `IsTightMeasureSet` and Prokhorov; Cylinders / Projective / ProjectiveFamilyContent; Kernel Ionescu-Tulcea; `FiniteDimensionalLaws`. Work must re-audit the actual pinned revision.

External candidate: RemyDegenne/`kolmogorov_extension4`, exact revision to be pinned by Work. Design-time inspected master commit `7d76e184c3d2138a2741baf923b57e9a01b9cf25`; Apache-2.0. Status remains EXTERNAL-CHECK until Work completes the required trust audit.

End of normative handoff.